

MCP servers

Seven MCP servers and one MCP client. For each one, this document explains what it does, how it keeps an agent inside safe limits, how I checked that it works, and how to connect it.

Arunkumar V · October 2026 · github.com/akv2011/mcp-servers

Server	What it does	Transport and auth
Outreach Guard	Email outreach over the Instantly API v2, with recipient checks and human approval before any send. Live on Vercel.	Streamable HTTP, OAuth 2.1 with Google sign-in, MCP 2026-07-28
Warehouse analytics	Analytics questions over DuckDB through a metrics layer and SQL that can only read.	stdio, or HTTP on localhost
Ad network reporting	Spend, installs, CPI, ROAS, anomalies and budget pacing across three ad networks in one schema.	stdio, or HTTP on localhost
Slack analytics bot	An MCP client. Answers Slack questions by calling the warehouse server and shows the SQL behind each number.	MCP client over stdio, Slack Socket Mode
Okta MCP server	Okta signs people in and Okta groups decide which tools each person can see and call.	Streamable HTTP, OAuth resource server with Okta tokens
DeFi MCP server	Lending, yield and trading data plus unsigned transactions for Aave V3, Morpho and Hyperliquid on six chains.	Streamable HTTP and SSE, API key
SMS to Cursor	Text a phone number and an agent acts in the Cursor editor through an MCP bridge.	stdio bridge, signed Twilio webhook
Market intelligence	A market analysis backend that gets prices, statements, news and filings through an MCP client.	stdio, MCP client inside the backend

What they have in common

- **The rule lives in the server, not the prompt.** Anything that matters is enforced in code a model cannot talk its way past: a middleware that sees every tool call, a database connection that cannot write, a scope check on each tool, a command allowlist run without a shell.
- **Tools describe themselves honestly.** Each tool carries annotations such as `readOnlyHint` and, where the framework allows, an output schema. A failure comes back as `isError` with a reason the model can read, so it never mistakes an error for data.
- **Tests are written to fail.** For the guarded servers I removed each guard on purpose and confirmed a test went red. A suite that stays green with the guard deleted proves nothing.
- **Checked in real clients.** On 2026-10-08 I connected each local server to Claude Code 2.1.294 and listed and called its tools in MCP Inspector. Codex CLI 0.156.1 accepted every config, and Gemini CLI 0.63.0 called a tool on each local server from a headless run.
- **No secrets in any repository.** Keys come from environment variables, and each repo ships only a `.env.example` with empty values.

Outreach Guard

Email outreach over the Instantly API v2, with a guard on every tool call.

github.com/akv2011/outreach-guard-mcp

outreach-guard-mcp.vercel.app

Python, FastMCP 4.0.11

MCP 2026-07-28

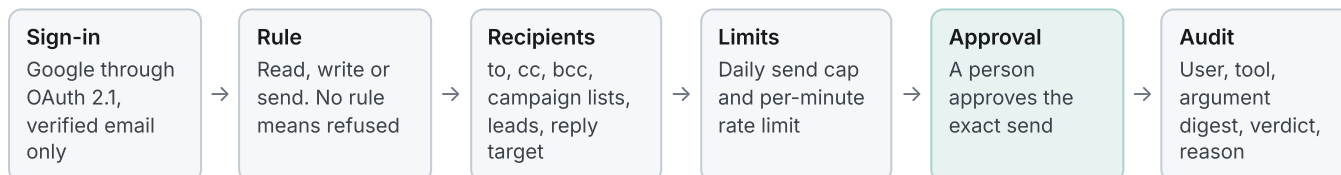
Vercel and Upstash Redis

An agent that can send email for you is useful right up to the moment it emails the wrong person. Outreach Guard lets an agent read campaigns, leads and the inbox freely. Creating campaigns needs an allowlisted account. Sending needs every recipient checked and a person to approve that exact send. Every decision goes into an audit log.

Without an Instantly API key it runs on demo data. The demo workspace answers with the field names from Instantly's published OpenAPI file and records sends instead of delivering them, so anyone can try the guard with no mailbox at risk. With a key, the same code talks to a live workspace.

Tool	Kind	Instantly endpoint
<code>list_campaigns</code>	read	GET <code>/api/v2/campaigns</code>
<code>campaign_analytics</code>	read	GET <code>/api/v2/campaigns/analytics</code>
<code>list_leads</code>	read	POST <code>/api/v2/leads/list</code>
<code>list_emails</code>	read	GET <code>/api/v2/emails</code> (the unibox)
<code>create_campaign</code>	write	POST <code>/api/v2/campaigns</code>
<code>add_leads</code>	write	POST <code>/api/v2/leads/add</code>
<code>activate_campaign</code>	send	POST <code>/api/v2/campaigns/{id}/activate</code> , after reading the campaign and every page of its leads
<code>reply_to_email</code>	send	POST <code>/api/v2/emails/reply</code> , after reading the original email to find who the reply goes to

WHAT HAPPENS ON EVERY CALL



How it keeps an agent in bounds

- **Who can do what.** Any signed-in Google account with a verified email can read. Writes need an email on `ALLOWED_EMAILS` . Sends need that and everything below.
- **Every recipient, wherever it hides.** The guard collects addresses from the to, cc and bcc fields, a campaign's cc and bcc lists, the leads being added, every lead of a campaign about to start, and the sender a reply goes back to. It normalizes case and display names, then checks each domain against an allowlist and against Instantly's block list. An address it cannot read is denied.
- **Hidden BCC.** An outside address in any BCC field is denied before approval is even asked for. This is the pattern behind the postmark-mcp incident, where a published MCP server quietly copied every outgoing email to its author.
- **Caps.** A daily cap on recipients per user, 5 by default, and a limit on tool calls per minute.
- **Fail closed.** A recipient lookup that fails denies the call. A tool registered without a rule stops the server from starting.
- **The server's own key.** The server calls Instantly with its own API key. The client's OAuth token never leaves the server.

Approval without server sessions

MCP 2026-07-28 made servers stateless, which raises a hard question for approvals. If no session holds the pending send, what stops a client from approving one send and replaying that approval on another? Outreach Guard uses the new multi round-trip flow. A send returns `resultType: "input_required"` with a small approval form. The client shows it, the person approves, and the client repeats the call with the answer and the `requestState` the server handed out.

That state is sealed with AES-GCM and bound to the user, a digest of the arguments and recipients, and a five-minute expiry. Change one recipient and the approval no longer fits. Clients still on the 2025-11-25 protocol cannot answer an input request, so they get a link to the web page instead. Approving there covers that exact call once, for ten minutes.

How it is built

The guard is one FastMCP middleware, so a new tool cannot skip it. The decision is a pure function, `check(rule, args, user, state, settings)`, driven by a table that maps each tool to its kind, the fields that hold recipients, and how to resolve them. Keeping the decision pure means every branch can be tested without a network. Redis holds counters, approvals, the audit log and OAuth client records, so every serverless instance sees the same state.

The web page offers the same tools through a Gemini agent, with a live decision log and a Guardrails panel. Each visitor can change their own demo rules (allowed domains, send cap, write access, whether sends need approval) and watch the guard respond. Turning approval off keeps every other check, and live mode ignores these overrides.

How I checked it

97 tests. The policy tests cover every branch of the decision function. The server tests run real HTTP in-process with bearer tokens, covering annotations, output schemas, the approval round trip, BCC exfiltration, request-state binding, rate limits and the audit log. Contract tests validate every demo response and every request the client sends against Instantly's OpenAPI file.

On the live deployment on 2026-10-08, an MCP client registered, signed in with Google, received a token bound to the `/mcp` resource, negotiated 2026-07-28, listed 8 tools, read campaigns, and was refused a `create_campaign` that BCC'd an outside address. In the web chat, the guard refused a reply with an outside BCC, held a clean reply for approval, sent it once on approve and sent nothing on reject.

Try it

```
claude mcp add --transport http outreach-guard \
https://outreach-guard-mcp.vercel.app/mcp
```

Or add the same URL as a custom connector in Claude and sign in with Google. The web page at the same address needs no client at all.

Known limits

Counters, the audit log and approval redemption use read-then-write on Redis, so two concurrent requests can race. Each spot is marked in the code with its fix. The block list and a campaign's leads are read up to 1,000 entries deep; past that the guard denies the send rather than guess.

Warehouse analytics

Analytics questions over a warehouse, without the power to change it.

github.com/akv2011/warehouse-analytics-mcp

Python, FastMCP 4

DuckDB and sqlglot

stdio or HTTP

Letting a model write SQL against a warehouse has two problems. It can write the wrong query, and it can write a harmful one. This server answers the first with a metrics layer and the second with three locks.

It ships with a generated warehouse for a mobile app, 20,000 users over 90 days of installs, sessions, purchases and ad revenue, so it runs with no setup. Pointing `WAREHOUSE_DB` at another DuckDB file swaps in real data. Every query decision is kept in an `audit://queries` resource.

Tool	What it does
<code>list_metrics</code>	Metrics with fixed definitions: installs, active users, sessions, purchase and ad revenue, paying users, day-1 and day-7 retention, ARPU, payer conversion
<code>query_metric</code>	One metric by day, week or month over a date range, split by country, platform or channel
<code>list_tables</code>	Tables with row counts and column types
<code>run_sql</code>	One read-only SELECT, for questions no metric answers

Metrics first

Ask two models for "revenue last week" and you can get two definitions of revenue. `query_metric` takes a metric name, a date range, a grain and dimensions, and builds the SQL from a YAML file of definitions. Every value is either checked against a fixed list or bound as a parameter, so there is nothing to inject. Ratio metrics such as ARPU are defined as one sum over another, so the numbers stay correct when grouped.

Three locks on `run_sql`

Each one is enough to stop a write on its own.

1. **The connection.** DuckDB opens the file read-only, with external access off, extension autoloading off and the configuration locked. A test sends `CREATE`, `DELETE`, `COPY ... TO`, `read_csv('/etc/hosts')`, `INSTALL`, `SET` and `ATTACH` straight to the connection, past every other check, and each one fails.
2. **The parser.** sqlglot parses the query. It must be exactly one `SELECT`, `UNION` or `CTE` query, with no write or settings nodes anywhere, no table functions, and only the published tables. The SQL rebuilt from the parsed tree is what runs, so anything hidden in comments is gone.
3. **Limits.** At most 1,000 rows, with a `truncated` flag, and a 10-second timeout.

A refused query comes back as `isError` with the reason, for example `query rejected: table functions are not allowed; query the published tables`.

How I checked it

55 tests. The parser faces 24 kinds of unsafe query, and 9 of them also go straight to the connection with the parser bypassed. Each metric is compared with hand-written SQL. Claude Code connects, and in MCP Inspector `query_metric` runs while `run_sql` refuses `read_csv('/etc/hosts')` with `isError`.

Try it

```
claude mcp add warehouse -- uv --directory \
/path/to/warehouse-analytics-mcp run warehouse-mcp
```

HTTP mode binds to localhost and has no sign-in of its own.

Ad network reporting

One vocabulary for spend, installs and return across ad networks.

github.com/akv2011/adnetwork-reporting-mcp

Python, FastMCP 4

stdio or HTTP

Every ad network reports differently. One sends cost in cents, one sends spend as a nested amount with a currency, and one uses US-style dates and calls installs "conversions". This server keeps one adapter per network that maps raw rows onto a single row type, so the tools and the model never see the differences.

It ships with 90 days of generated data for three demo networks, each with its own raw format, eight campaigns and eight countries. Two incidents are planted on purpose so the anomaly tool has known answers. Video network cost is multiplied by four on 2026-09-10, and search installs drop to 15% of normal on 2026-08-20.

Tool	What it returns
<code>list_networks</code>	Networks, their campaigns and the dates covered
<code>compare_networks</code>	Spend, installs, CPI, day-7 ROAS and share of spend per network
<code>get_report</code>	The same measures grouped by any of network, campaign, country and day
<code>find_anomalies</code>	Campaign-days where spend, installs or CPI sit far from that campaign's own previous 14 days
<code>budget_pacing</code>	Spend so far against each campaign's monthly budget, projected to month end, marked under, on track or over

How the numbers stay honest

- **One currency.** A report in anything other than USD is refused rather than quietly added to USD totals.
- **Ratios from totals.** CPI and ROAS are computed from summed spend, installs and revenue, never by averaging daily ratios, so grouped numbers add up.
- **Bounded inputs.** Networks, dimensions, metrics and months come from fixed lists. Date ranges are capped at 92 days, and anomaly thresholds must sit between 2 and 10 standard deviations. A bad input returns `isError` with the reason.
- **Each campaign against itself.** Anomalies are scored against the campaign's own previous 14 days, so a campaign that is always expensive is not flagged just for being expensive.

How I checked it

17 tests: each adapter against its raw format, the currency check, totals adding up across groupings, both planted anomalies ranking first while a quiet window stays quiet, the pacing arithmetic, input limits and the MCP surface. Breaking an adapter or the anomaly baseline makes them fail. Claude Code connects, and MCP Inspector lists the five tools and runs `budget_pacing`.

Try it

```
claude mcp add adnet -- uv --directory \
/path/to/adnetwork-reporting-mcp run adnet-mcp
```

Real networks

The data is generated. To report on real networks, add a fetch function and an adapter for each one; the reports and tools stay the same.

Slack analytics bot

An MCP client. Questions asked in Slack, answered by the warehouse server.

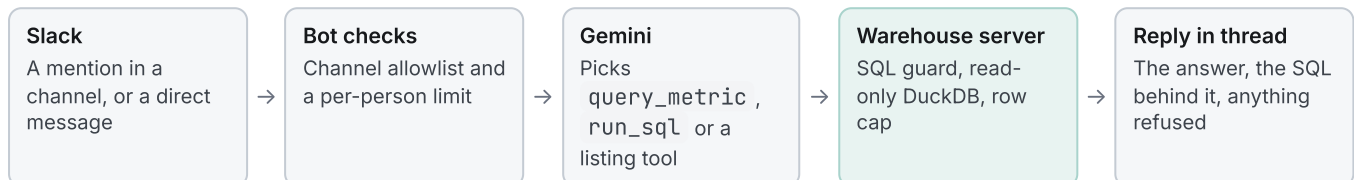
github.com/akv2011/slack-analytics-bot

Python, Slack Bolt, FastMCP client

Gemini 3.8 Flash

This one sits on the other side of the protocol. Mention the bot in a channel or send it a direct message, and it starts the warehouse server over stdio, hands its tool list to Gemini, runs the tool calls Gemini asks for, and replies in the thread.

The bot never opens the database. Every guard on SQL stays in the warehouse server, so nothing typed into Slack can get past it, however the question is phrased.



How it behaves

- **Every number shows its query.** The reply carries the SQL that produced the answer, so anyone in the channel can check it.
- **Refusals stay visible.** If the server rejects a query, the reply says what was refused instead of quietly answering something else.
- **No guessing.** The model is told never to state a number no tool returned, and the loop stops after six steps with "I could not finish" rather than an estimate.
- **Where and how often.** A channel allowlist decides where it answers (direct messages always work), and each person gets 20 questions an hour by default.
- **No loops, no leaks.** Bot messages and edits are ignored, so it cannot answer its own replies. When something breaks, the person gets a short message and the details go to the server log, not the channel.

How I checked it

From the terminal, asked for day-7 retention by platform in August 2026, it answered 24.66% on iOS and 22.90% on Android. A direct query on the same data gave the same two numbers.

12 tests. The agent tests run the tool loop against a real in-process warehouse server with a scripted model, and check that Gemini's own turns go back unchanged so its thought signatures survive the loop. The Slack tests call the event handler with a fake reply function. Removing the allowlist, the rate limit, the thread reply, failure recording or the step limit each fails a test. It has not yet run against a live Slack workspace; the app manifest and setup steps are in the README.

```
uv run --env-file .env ask-warehouse "What was day-7 retention by platform in August 2026?"
uv run --env-file .env slack-analytics-bot      # Socket Mode, needs no public URL
```

Okta MCP server

Company sign-in instead of a shared API key, with Okta groups deciding which tools you get.

github.com/akv2011/okta-mcp-poc

Python, FastMCP 4

OAuth resource server

Docker Compose

Most internal MCP servers start with a shared API key, which says nothing about who is calling. This server is an OAuth resource server instead. Okta signs people in, and their Okta groups decide which tools they can see and call. Moving someone between groups in Okta changes their tools with no code change and no restart. No secret sits in any config file.

How sign-in works

1. A call without a token gets 401 with a WWW-Authenticate header pointing at the server's protected resource metadata.
2. That metadata names the Okta authorization server and the scopes this server understands.
3. The client opens Okta in the browser, the person signs in, and the client receives a token whose audience is this server, through PKCE with a resource parameter.
4. On every call the server checks the token's signature against Okta's published keys, along with its issuer, expiry and audience, then reads the groups and scp claims.

Tool	Scope	Okta groups
hello	mcp:tools.read	mcp-readers, mcp-admins
whoami	mcp:tools.read	mcp-readers, mcp-admins
admin_stats	mcp:tools.admin	mcp-admins

A reader's tools/list shows only hello and whoami. The admin tool does not appear at all, which is better than showing it and refusing the call.

Setting up Okta

scripts/okta_setup.py configures an Okta org through its admin API: the audience, both scopes, a groups claim filtered to mcp-*, the two groups, a public OIDC app with redirect URIs for Claude Code, Codex and the Claude apps, and policy rules that grant the admin scope only to admins. It is safe to run twice, and the API token it uses can be revoked as soon as it finishes.

How I checked it

11 tests run the real HTTP stack in-process with tokens signed locally in Okta's shape, so they need no Okta org. They cover the 401 challenge, the metadata document, five kinds of bad token (wrong audience, wrong issuer, expired, wrong signing key, missing scope), a reader seeing two tools and being refused the admin tool, an admin getting all three, and whoami echoing the token's claims. Granting the admin tool to readers, advertising the wrong issuer, or removing auth each fails a test.

```
docker compose up -d --build
claude mcp add --transport http --client-id <okta client id> --callback-port 8765 \
  okta-mcp http://localhost:8080/mcp
```

DeFi MCP server

DeFi data and unsigned transactions, with no tool able to move funds.

github.com/akv2011/API_mcp_servers_builds

TypeScript, NestJS

@rekog/mcp-nest

Streamable HTTP and SSE

A NestJS service that exposes lending, yield and trading data for Aave V3, Morpho and Hyperliquid across Ethereum, Base, Arbitrum, Optimism, Sonic and Mode, plus builders for Aave, Morpho and ERC-20 approval transactions. The same service also serves a REST API with Swagger docs.

No tool signs or sends anything. The `generate_*` tools return unsigned transaction data for the user's own wallet to sign, so the server never holds a key that can move money. All 16 tools are marked read-only.

Tools	What they return
<code>get_lending_positions</code> , <code>get_lending_markets</code> , <code>get_yield_opportunities</code>	A wallet's lending positions, lending markets and yield opportunities, filterable by chain, protocol and asset
<code>get_hyperliquid_positions</code> , <code>get_hyperliquid_open_orders</code>	A Hyperliquid account's positions and open orders
<code>get_token_info</code> , <code>get_token_balances</code> , <code>get_wallet_balance</code>	Token metadata and prices, and wallet balances
<code>generate_aave_supply_tx</code> and the withdraw, borrow and repay builders	Unsigned Aave V3 transactions
<code>generate_morpho_borrow_tx</code> and the vault deposit and withdraw builders	Unsigned Morpho transactions
<code>generate_token_approval_tx</code>	An unsigned ERC-20 approval with a gas limit from an estimate and EIP-1559 fee caps

How it is protected

- **A key on every route.** `/mcp` , `/sse` and the REST API all need an API key, sent as a bearer token, an `x-api-key` header, or a query parameter for clients that cannot set headers. A local key is compared in constant time.
- **Nothing secret in logs or errors.** A query-string key is moved into a header before any handler or library sees the URL. RPC URLs carry the provider's key and viem copies them into its errors, so errors leave the RPC layer with only the URL's origin.
- **Errors look like errors.** A failed tool call sets `isError` on the result, so a model can tell a failure from data.
- **Docs that cannot drift.** The tool reference is generated from the server's own `tools/list` .

How I checked it

Claude Code connects. MCP Inspector lists all 16 tools and runs live calls, including Hyperliquid positions and a mainnet USDC approval priced at 0.71 gwei with a 56,361 gas limit. Requests with no key, or with a wrong one, get 401.

Try it

```
pnpm install && pnpm build && node dist/main
claude mcp add --transport http defi-mcp \
  http://localhost:3000/mcp \
  -H "Authorization: Bearer $MCP_API_KEY"
```

Known limits

It runs on mcp-nest 1.6, which negotiates MCP 2025-06-18. Version 2.x targets the current spec but replaces the module API, so each tool class needs porting. Hosted API keys are stored in plaintext; hashing them needs a table migration.

SMS to Cursor

Text a phone number and an agent works in your editor.

github.com/akv2011/MCP_Automated_Siri_cursor_control

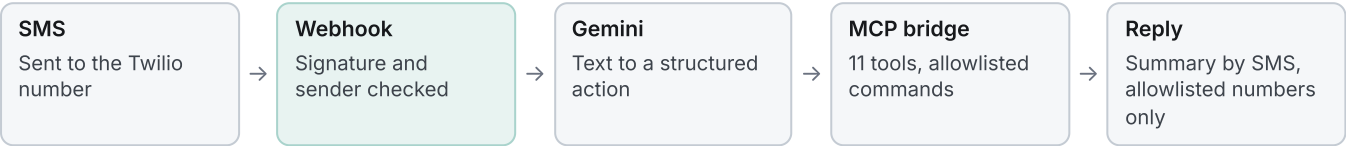
Python, Flask, FastMCP 4

Twilio

MCP 2026-07-28 over stdio

A Twilio webhook receives a text message, Gemini turns it into a structured action, and the action reaches Cursor through an MCP bridge, with UI automation as a fallback. A summary comes back by text.

Anything that turns a text message into actions on a machine is remote code execution unless every hop is checked. Most of the work here went into those checks.



Entry point	Check
POST /sms	A valid Twilio signature for the public URL Twilio called, and the sender must be on ALLOWED_SENDERS
Every other route	An admin token, sent as a header or once as a query parameter that becomes an HttpOnly, SameSite=Strict cookie. With no token configured, these routes stay closed
Any outgoing SMS	The recipient must be allowlisted, and every send goes through one function, whether a route or an MCP tool triggered it
run_command tool	Parsed into arguments and run without a shell. Only ls , grep , find without -exec or -delete , and read-only git
HTML responses	User text is escaped
Network	The app and the UI bridge listen on 127.0.0.1, and debug mode needs an explicit opt-in

How I checked it

35 tests cover the webhook signature and sender checks, the admin token, the SMS allowlist on routes and MCP tools, HTML escaping, and the command parser, including that `ls ; touch pwned` creates nothing. Turning the guards off makes 24 of them fail. Claude Code connects to the bridge, and MCP Inspector lists all 11 tools.

Try it

```
python src/app.py      # webhook on 127.0.0.1:5000
ngrok http 5000        # set PUBLIC_URL to its https URL
```

Known limits

The UI automation fallback uses keyboard shortcuts written for Cursor on Windows. The MCP bridge, the webhook and the guards run on any OS.

Market intelligence

A backend that gets its market data through an MCP client.

github.com/akv2011/Market_intelligence_MCP

Python, FastAPI, FastMCP 4

MCP 2026-07-28 over stdio

A FastAPI service that answers market questions in two stages. Gemini first works out the ticker and what kind of question it is. The service then fetches data, adds Google Search grounding, and has Gemini write the answer with citations. Answers stream over server-sent events, and conversation history is kept in SQLite so follow-up questions keep their context.

The market data comes from an MCP server over the Financial Datasets API, adapted from the open-source `financial-datasets/mcp-server` (MIT) and moved to FastMCP 4. The backend starts it over stdio and calls it through an MCP client for every query that needs data. The same server works unchanged in any desktop MCP client.

Area	Tools
Statements	<code>get_income_statements</code> , <code>get_balance_sheets</code> , <code>get_cash_flow_statements</code>
Stocks	<code>get_current_stock_price</code> , <code>get_historical_stock_prices</code>
News and filings	<code>get_company_news</code> , <code>get_sec_filings</code>
Crypto	<code>get_available_crypto_tickers</code> , <code>get_crypto_prices</code> , <code>get_historical_crypto_prices</code> , <code>get_current_crypto_price</code>

How I checked it

Claude Code connects to the MCP server, and MCP Inspector lists all 11 tools and runs them. Run locally on 2026-10-08, the backend answered live market questions on `gemini-3.8-flash`.

One fix here belongs to the same theme as the rest of this document. The API used to return provider errors to the caller word for word, and Google's error for a suspended key quotes the key itself. Errors are now logged on the server and the caller gets a fixed message. A test feeds the client an error that carries a fake key and fails if the key comes back.

```
claude mcp add market-intel -e FINANCIAL_DATASETS_API_KEY=<your key> -- \
  uv --directory /path/to/Market_intelligence_MCP/mcp-server-main run server.py

python -m uvicorn api.main:app --port 8000
curl -X POST http://localhost:8000/query \
  -H "Content-Type: application/json" -d '{"message": "What is NVIDIA stock price?"}'
```